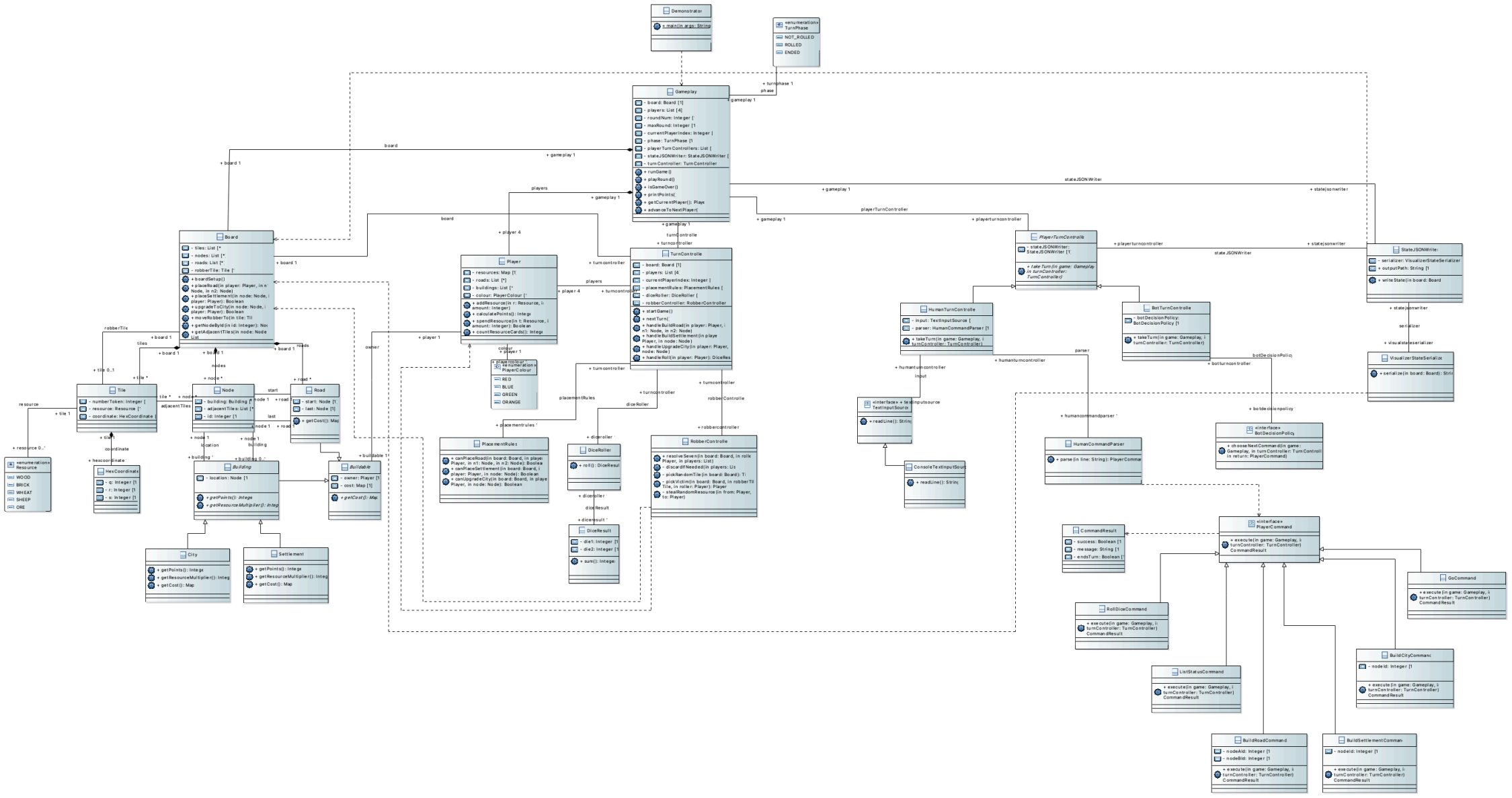
Demonstrator
+ main(in args: String
«enumeration» TurnPhase
NOT_ROLLED
ROLLED
ENDED
Gameplay
- board: Board [1]
- players: List [4]
- roundNum: Integer [
- maxRound: Integer [1
- currentPlayerIndex: Integer [
- phase : TurnPhase [1
- playerTurnControllers: List [
- stateJSONWriter: StateJSONWriter [
- turnController: TurnController
+ runGame()
+ playRound()
+ isGameOver()
+ printPoints(
+ getCurrentPlayer(): Playe
+ advanceToNextPlayer(
Board
- tiles: List [*
- nodes: List [*
- roads: List [*
- robberTile: Tile [
+ boardSetup()
+ placeRoad(in player: Player, in n
Node, in n2: Node)
+ placeSettlement(in node: Node, i
player: Player): Boolean
+ upgradeToCity(in node: Node, i
player: Player): Boolean
+ moveRobberTo(in tile: Til
+ getNodeById(in id: Integer): Noc
+ getAdjacentTiles(in node: Node
List
Player
- resources: Map [1
- roads: List [*]
- buildings: List [*
- colour: PlayerColour [
+ addResource(in r: Resource, i
amount: Integer)
+ calculatePoints(): Intege
+ spendResource(in t: Resource, i
amount: Integer): Boolean
+ countResourceCards(): Intege
TurnController
- board: Board [1]
- players: List [4
- currentPlayerIndex: Integer [
- placementRules: PlacementRules [
- diceRoller: DiceRoller [
- robberController: RobberController
+ startGame()
+ nextTurn()
+ handleBuildRoad(in player: Player, i
n1: Node, in n2: Node)
+ handleBuildSettlement(in playe
Player, in node: Node)
+ handleUpgradeCity(in player: Player,
node: Node)
+ handleRoll(in player: Player): DiceRes
PlayerTurnControlle
- stateJSONWriter:
StateJSONWriter [1
+ takeTurn(in game: Gameplay
in turnController:
TurnController)
StateJSONWrite
- serializer: VisualizerStateSerializer
+ outputPath: String [1
+ writeState(in board: Board
HumanTurnControlle
- input: TextInputSource [
- parser: HumanCommandParser [1
+ takeTurn(in game: Gameplay, i
turnController: TurnController)
BotTurnControlle
- botDecisionPolicy:
BotDecisionPolicy [1
+ takeTurn(in game: Gameplay, i
turnController: TurnController)
VisualizerStateSerializ
+ serialize(in board: Board): Strin
«interface» + textinputsource
TextInputSource
+ readLine(): Strin
ConsoleTextInputSour
+ readLine(): Strin
HumanCommandParser
+ parse(in line: String): PlayerComma
«interface»
BotDecisionPolicy
+ chooseNextCommand(in game:
Gameplay, in turnController: TurnControll
in return: PlayerCommand)
«interface»
PlayerCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
CommandResult
- success: Boolean [1
- message: String [1
- endsTurn: Boolean [
GoCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
RollDiceCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
ListStatusCommand
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildCityCommanc
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildRoadCommand
- nodeAId: Integer [1
- nodeBId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
BuildSettlementComman
- nodeId: Integer [1
+ execute(in game: Gameplay, i
turnController: TurnController)
CommandResult
Tile
- numberToken: Integer [
- resource: Resource [*
- coordinate: HexCoordinate
Node
- building: Building [
- adjacentTiles: List [*
- id: Integer [1
Road
- start: Node [1]
- last: Node [1]
+ getCost(): Map
«enumeration» Resource
WOOD
BRICK
WHEAT
SHEEP
ORE
HexCoordinate
- q: Integer [1]
- r: Integer [1]
- s: Integer [1]
Building
- location : Node [1
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
Buildable
- owner: Player [1]
- cost: Map [1]
+ getCost(): Map
City
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map
Settlement
+ getPoints(): Intege
+ getResourceMultiplier(): Integ
+ getCost(): Map
«enumeration» PlayerColour
RED
BLUE
GREEN
ORANGE
PlacementRules
+ canPlaceRoad(in board: Board, in playe
Player, in n1: Node, in n2: Node): Boolea
+ canPlaceSettlement(in board: Board, i
player: Player, in node: Node): Boolean
+ canUpgradeCity(in board: Board, in playe
Player, in node: Node): Boolean
DiceRoller
+ roll(): DiceResul
DiceResult
- die1: Integer [1
- die2: Integer [1
+ sum(): Intege
RobberController
+ resolveSeven(in board: Board, in rolle
Player, in players: List)
- discardIfNeeded(in players: Lis
- pickRandomTile(in board: Board): Ti
- pickVictim(in board: Board, in robberTil
tile, in roller: Player): Player
- stealRandomResource(in from: Player,
to: Player)
+ board 1
board
+ gameplay 1
+ turnphase 1
phase
+ gameplay 1
stateJSONWriter
+ statejsonwriter
+ gameplay 1
+ gameplay 1
playerTurnController
+ gameplay 1
+ playerturncontroller
+ gameplay 1
turnControlle
+ turncontroller
+ player 4
+ turncontroller
+ turncontroller
+ playerturncontroller
+ statejsonwriter
state JSONWriter
+ statejsonwriter
serializer
+ visualizerserializer
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
+ board 1
robberTile
tiles
nodes
roads
+ tile 0..1
+ tile *
+ node *
+ road *
+ tile 1
adjacentTiles
+ node *
start
+ road 1
last
+ node 1
+ road 1
resource
+ resource 0..*
+ start
coordinate
+ hexcoordinate
+ node 1
location
+ building *
+ node 1
building
+ building 0..*
+ buildable 1
owner
+ player 1
colour
+ player 1
+ player colour
players
+ player 4
+ turncontroller
+ turncontroller
placementRules
+ turncontroller
diceRoller
+ turncontroller
robberControlle
+ placementrules *
+ diceroller *
+ diceroller
diceResult
+ diceresult *
+ robbercontroller
+ humanturncontroller
input
+ humanturncontroller
parser
+ humancommandparser *
+ botdecisionpolicy *
botDecisionPolicy
+ botturncontroller